



دانشگاه تهران

پردیس دانشکده های فنی

دانشکده مهندسی برق و کامپیوتر

گزارش تمرین شماره یک

عنوان درس: مبانی رایانش توزیع شده

امین آقاکثیری – 810101381

مهدی نائینی – 810101536

بهار 1405

بخش اول: ارتباط بین دو پردازش مستقل با Pipe

مقدمه

هدف از این بخش تمرین آشنایی با مفهوم ارتباط بین فرآیندها در سیستم عامل های مبتنی بر لینوکس است. یکی از کاربردی ترین روش‌های IPC در سیستم عامل لینوکس استفاده از Pipe است.

در این پروژه یک سرویس محاسباتی ساده پیاده سازی شده است که از دو فرآیند مستقل تشکیل می شود. فرآیند اول که Interface نام دارد نقش رابط کاربر را ایفا می کند و مسئول دریافت درخواست های کاربر و نمایش نتایج است. فرآیند دوم که Worker نام دارد به صورت مداوم در حال اجرا بوده و وظیفه پردازش درخواست های دریافتی و ارسال پاسخ مناسب را بر عهده دارد. ارتباط میان این دو فرآیند با استفاده از Named Pipe انجام می شود.

پیاده سازی این پروژه با استفاده از زبان برنامه‌نویسی Go انجام شده و تنها از کتابخانه های استاندارد این زبان استفاده شده است. هدف اصلی طراحی مناسب برنامه و مدیریت صحیح است که بتواند چندین درخواست متوالی را بدون نیاز به راه اندازی مجدد پردازش کند.

طراحی کلی سیستم

معماری سیستم از دو برنامه مستقل تشکیل شده است که هر کدام در یک فرآیند جداگانه اجرا می شوند. این دو برنامه از طریق دو FIFO مجزا با یکدیگر ارتباط برقرار می‌کنند. اولین FIFO برای ارسال درخواست ها از Interface به Worker استفاده می شود و FIFO دوم برای ارسال پاسخ‌ها از Worker به Interface به کار می رود.

فرآیند Worker پس از اجرا ابتدا بررسی می کند که FIFO های مورد نیاز وجود دارند یا خیر. در صورتی که این فایل ها موجود نباشند، Worker آن ها را ایجاد می کند. سپس Worker وارد یک حلقه بی نهایت می شود و به صورت مداوم منتظر دریافت درخواست های جدید از طریق FIFO مربوط به درخواست ها باقی می ماند. هر درخواست پس از دریافت پردازش شده و نتیجه آن از طریق FIFO پاسخ به Interface ارسال می شود.

در سمت دیگر، فرآیند Interface ابتدا منتظر می‌ماند تا FIFO های مورد نیاز توسط Worker ایجاد شوند. پس از آماده شدن کانال های ارتباطی، Interface ورودی کاربر را از ترمینال دریافت کرده و آن را به Worker ارسال می کند. سپس پاسخ Worker را که به صورت JSON ارسال شده است دریافت کرده و نتیجه را برای کاربر نمایش می دهد.

ساختار پیام ها و پروتکل ارتباطی

هر درخواست در قالب متن یک خطی شامل سه بخش ارسال می شود. بخش اول نام عملیات محاسباتی است و دو بخش بعدی اعداد ورودی هستند. این سه بخش توسط فاصله از یکدیگر جدا می شوند و در انتهای رشته یک کاراکتر newline قرار می گیرد. به عنوان مثال درخواست جمع دو عدد به صورت ADD 5 7 ارسال می شود.

Worker پس از دریافت این رشته ابتدا آن را تجزیه کرده و اجزای آن را استخراج می کند. در صورتی که فرمت درخواست معتبر باشد، عملیات مربوطه انجام شده و نتیجه محاسبه می شود.

پاسخ Worker به صورت JSON ارسال می شود. در حالت موفقیت پاسخ شامل وضعیت OK و مقدار نتیجه است. در صورت بروز خطا وضعیت ERROR همراه با توضیح خطا ارسال می شود. برای مثال یک پاسخ موفق می تواند به شکل زیر باشد:

```
{"status": "OK", "result": 12}
```

و در صورت بروز خطا پاسخ به صورت زیر خواهد بود:

```
{"status": "ERROR", "error": "Division by zero"}
```

در نهایت Interface این پیام JSON را دریافت کرده و سپس آن را به شکل مناسب برای کاربر نمایش می دهد.

پیاده سازی فرآیند Worker

فرآیند Worker وظیفه پردازش تمام درخواست های محاسباتی را بر عهده دارد. این برنامه پس از اجرا ابتدا وجود FIFO های مورد نیاز را بررسی می کند و در صورت نبود آن ها را ایجاد می کند. سپس فایل مربوط به FIFO درخواست ها در حالت خواندن و FIFO پاسخ ها در حالت نوشتن باز می شود.

پس از آماده شدن کانال‌های ارتباطی، Worker وارد یک حلقه پردازشی می شود. در این حلقه برنامه منتظر می ماند تا FIFO یک خط جدید از درخواست ها را دریافت کند. هر خط دریافت شده به عنوان یک درخواست مستقل در نظر گرفته می شود. ابتدا رشته دریافتی تجزیه شده و نوع عملیات و اعداد ورودی استخراج می شوند. سپس با توجه به نوع عملیات، محاسبه مربوطه انجام می شود.

عملیات هایی که در این پروژه پیاده سازی شده اند شامل جمع، تفریق، ضرب، تقسیم، باقی مانده تقسیم، توان، بیشینه و کمینه هستند. نتیجه محاسبه پس از انجام عملیات در قالب JSON در FIFO پاسخ نوشته می شود.

Worker پس از پردازش هر درخواست همچنان فعال باقی می ماند و برای دریافت درخواست های بعدی آماده است. به این ترتیب برنامه می تواند درخواست متوالی را بدون نیاز به راه اندازی مجدد مدیریت کند.

پیاده سازی فرآیند Interface

فرآیند Interface به عنوان نقطه تعامل کاربر با سیستم عمل می کند. این برنامه پس از اجرا ابتدا بررسی می کند که FIFO های مورد نیاز در سیستم وجود دارند یا خیر. اگر Worker هنوز اجرا نشده باشد آنگاه Interface منتظر ایجاد این FIFO ها می ماند.

پس از آماده شدن کانال های ارتباطی، Interface شروع به دریافت ورودی از کاربر می کند. هر دستور پس از دریافت در FIFO درخواست نوشته شده و سپس Interface منتظر دریافت پاسخ از FIFO پاسخ می ماند.

پس از دریافت پاسخ در صورتی که وضعیت پاسخ OK باشد، مقدار نتیجه به کاربر نمایش داده می شود. در غیر این صورت پیام خطا چاپ می شود.

همچنین Interface در صورت قطع شدن ارتباط با FIFO ها تلاش می کند دوباره آن ها را باز کند. این موضوع باعث می شود برنامه در برابر قطع شدن ناگهانی Worker مقاوم باشد.

مدیریت خطا

یکی از مهم‌ترین موارد بررسی صحت ورودی کاربر است. اگر تعداد پارامترهای درخواست صحیح نباشد یا عملوندها قابل تبدیل به عدد نباشند، Worker یک پیام خطای مناسب تولید می کند.

در عملیات هایی مانند تقسیم و باقی مانده تقسیم نیز حالت تقسیم بر صفر بررسی شده و در صورت وقوع این حالت پیام خطای مناسب ارسال می شود. علاوه بر این اگر کاربر عملیاتی را وارد کند که در سیستم تعریف نشده باشد، Worker آن را به عنوان عملیات نامعتبر تشخیص داده و پیام خطا بر می گرداند.

در سطح ارتباط بین فرآیندها نیز خطا های مربوط به باز کردن FIFO ها، خواندن داده ها و نوشتن پاسخ ها مدیریت شده اند. در صورت بروز چنین خطا هایی پیام مناسب در log ثبت شده و برنامه تلاش می کند به کار خود ادامه دهد.

نحوه اجرا و مثال های عملی

برای اجرای این برنامه ابتدا Worker اجرا می شود تا FIFO های مورد نیاز ایجاد شوند. پس از آن Interface اجرا می شود و کاربر قادر خواهد بود درخواست های محاسباتی خود را وارد کند. در ادامه نمونه ای از اجرای سیستم آمده است:

```
$ ./worker
[WORKER] 2026/04/30 16:08:56 Worker process started
[WORKER] 2026/04/30 16:08:59 Worker Listening on ./temp/request.fifo
[WORKER] 2026/04/30 16:09:07 Received request: "ADD 3 6\n"
[WORKER] 2026/04/30 16:09:07 Sent response: {"status":"OK","result":9}
[WORKER] 2026/04/30 16:09:15 Received request: "DIV 20 0\n"
[WORKER] 2026/04/30 16:09:15 Sent response: {"status":"ERROR","error":"Division by zero"}
[WORKER] 2026/04/30 16:09:29 Received request: "MUL a b\n"
[WORKER] 2026/04/30 16:09:29 Sent response: {"status":"ERROR","error":"Invalid operand"}
[WORKER] 2026/04/30 16:09:49 Received request: "OP 2 3\n"
[WORKER] 2026/04/30 16:09:49 Sent response: {"status":"ERROR","error":"Invalid operation"}
[WORKER] 2026/04/30 16:09:59 Received request: "ADD 7\n"
[WORKER] 2026/04/30 16:09:59 Sent response: {"status":"ERROR","error":"Invalid arguments"}
[WORKER] 2026/04/30 16:10:47 Received request: "MUL 8 9\n"
[WORKER] 2026/04/30 16:10:47 Sent response: {"status":"OK","result":72}
```

```
$ ./interface
[INTERFACE] 2026/04/30 16:08:59 Interface process started
Enter your requests:
> ADD 3 6
OK 9.000000
> DIV 20 0
ERROR Division by zero
> MUL a b
ERROR Invalid operand
> OP 2 3
ERROR Invalid operation
> ADD 7
ERROR Invalid arguments
> MUL 8 9
OK 72.000000
```

جمع بندی

در این بخش از تمرین یک سرویس محاسباتی ساده با استفاده از ارتباط بین فرآیند ها از طریق Named Pipe پیاده سازی شد که در آن یک فرآیند به عنوان سرویس دهنده و فرآیند دیگر به عنوان رابط کاربر عمل می کند. نتیجه نهایی سیستمی است که می تواند درخواست های متعدد کاربر را به صورت پایدار پردازش کند و پاسخ مناسب را بازگرداند و خطا های متعدد را مدیریت کند.

بخش دوم: هم زمانی، زمان بندی و مشاهده ی اثرات Context Switching

مقدمه

در بخش دوم این تمرین هدف بررسی هم زمانی در زبان Go و هزینه های context switching بر یک برنامه ی چند گوروتینه است. در این قسمت تمرکز روی اجرای تعداد زیادی goroutine داخل یک فرآیند واحد است. در این بخش قرار است با تغییر سه پارامتر اصلی یعنی تعداد goroutine ها، نوع بار کاری و مقدار GOMAXPROCS رفتار زمان بندی Go و تاثیر این تغییرات بر معیار هایی مانند زمان کل اجرا، throughput و latency تسک ها مشاهده و تحلیل شود.

در این برنامه مجموعه ای از آزمایش ها را برای ترکیب های مختلف این پارامترها اجرا می کنیم و خروجی هر آزمایش را در یک فایل CSV ذخیره می نماییم. این فایل به عنوان ورودی برای تولید جدول ها و نمودار های بخش نتایج گزارش استفاده خواهد شد.

نحوه ی طراحی برنامه

برنامه به گونه ای طراحی شده که سه پارامتر کلیدی را در فضایی از حالت ها تغییر می دهد مثل تعداد goroutine ها، نوع بار کاری و مقدار GOMAXPROCS. برای هر ترکیب از این پارامتر ها، مجموعه ای از تسک ها اجرا می شود و معیار های مورد نیاز محاسبه و ثبت می گردد.

ساختار برنامه

برنامه دارای سه ساختار داده ی اصلی به نام ExperimentConfig برای توصیف پیکربندی هر آزمایش، Result برای نگه داشتن نتایج محاسباتی هر آزمایش و GoroutineStatistics برای جمع آوری آمار در سطح هر goroutine است. همچنین سه لیست برای پارامتر ها تعریف شده است. یک لیست برای نوع بار کاری که شامل cpu و mixed است. یک لیست دیگر که مقادیر GOMAXPROCS شامل 1، 2 و مقدار بازگشتی از runtime.NumCPU را نگه می دارد و لیست سوم که تعداد goroutine ها شامل مقادیر 1، 2، 4، 8، 16، 32 و 64 را در خود نگه می دارد. برای هر ترکیب یک ExperimentConfig ساخته می شود و تابع

runExperiment با این پیکربندی فراخوانی می شود. در پایان همه ی نتایج در فایل results/experiment_results.csv ذخیره می شوند.

پیاده سازی بار های کاری

دو سناریو برای نوع بار کاری پیاده سازی شده است. سناریوی اول CPU محور است که در آن goroutine ها تنها محاسبات عددی انجام می دهند بدون عملیات انتظار یا رقابت بر سر منابع مشترک. سناریوی دوم به صورت ترکیبی است. یعنی در آن علاوه بر محاسبه، هر goroutine روی یک mutex مشترک قفل را در اختیار می گیرد و در انتها یک تاخیر کوتاه با استفاده از time.Sleep ایجاد می کند. به این ترتیب تفاوت رفتار سیستم در مواجهه با بار های کاری متفاوت قابل مشاهده خواهد شد.

طراحی goroutine ها و جمع آوری داده ها

برای هر goroutine یک وظیفه تعریف شده است که در آن یک تعداد ثابت تسک پشت سر هم انجام می شود. این وظیفه توسط تابع worker پیاده‌سازی شده است. برای هر تسک زمان شروع ثبت می شود و بعد از پایان تسک، latency آن محاسبه و به مجموع latency ها افزوده می شود. در کنار آن مربع latency نیز محاسبه می شود تا بتوان در مراحل بعدی واریانس و انحراف معیار را محاسبه کرد. همچنین حداقل و حداکثر latency نیز در صورت لزوم به روزرسانی می شوند. پس از اتمام تمام تسک های یک goroutine، worker داده های جمع آوری شده خود را از طریق یک کانال اشتراکی به goroutine اصلی ارسال می کند. پس از پایان هر آزمایش، زمان پایان ثبت می‌شود و زمان کل اجرا به دست می‌آید. throughput به صورت نسبت تعداد کل تسک ها به زمان کل اجرا می شود. در نهایت تمام این مقادیر به عنوان خروجی برگردانده می شود.

ذخیره سازی نتایج

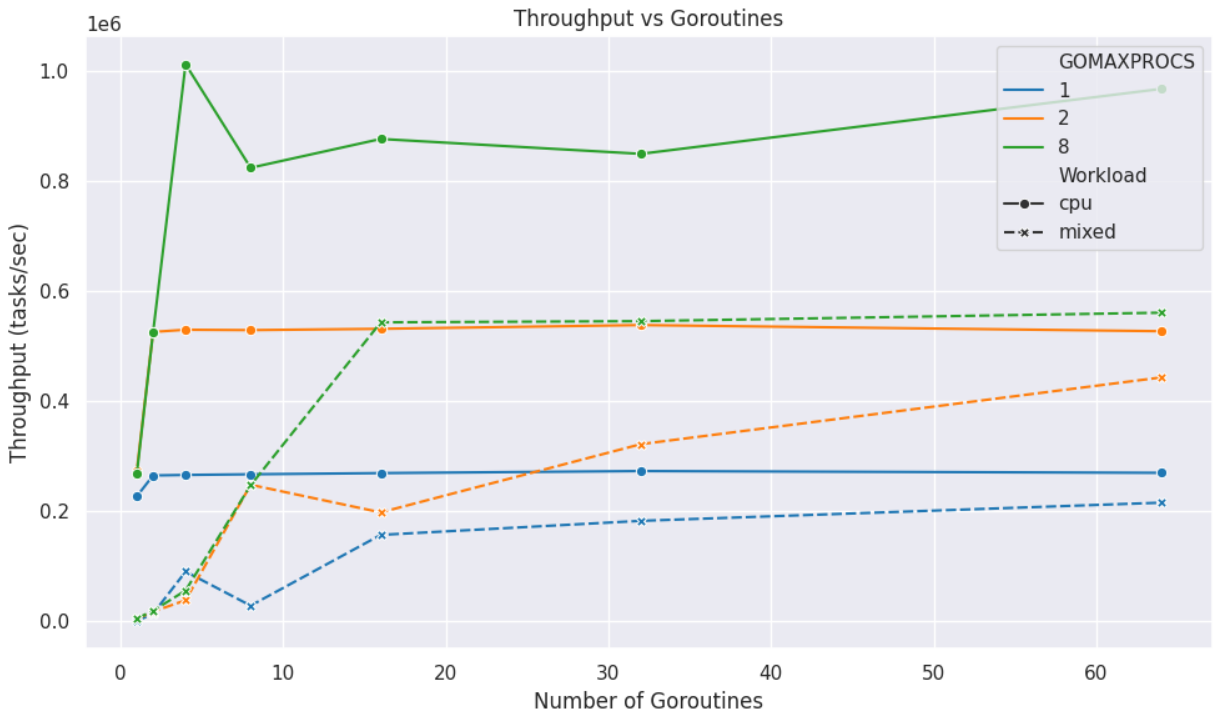
پس از اجرای تمام آزمایش ها و برای تحلیل راحت تر این نتایج و ترسیم نمودارها، داده ها در قالب CSV در پوشه ی results ذخیره می شوند.

نتایج آزمایش ها و تحلیل

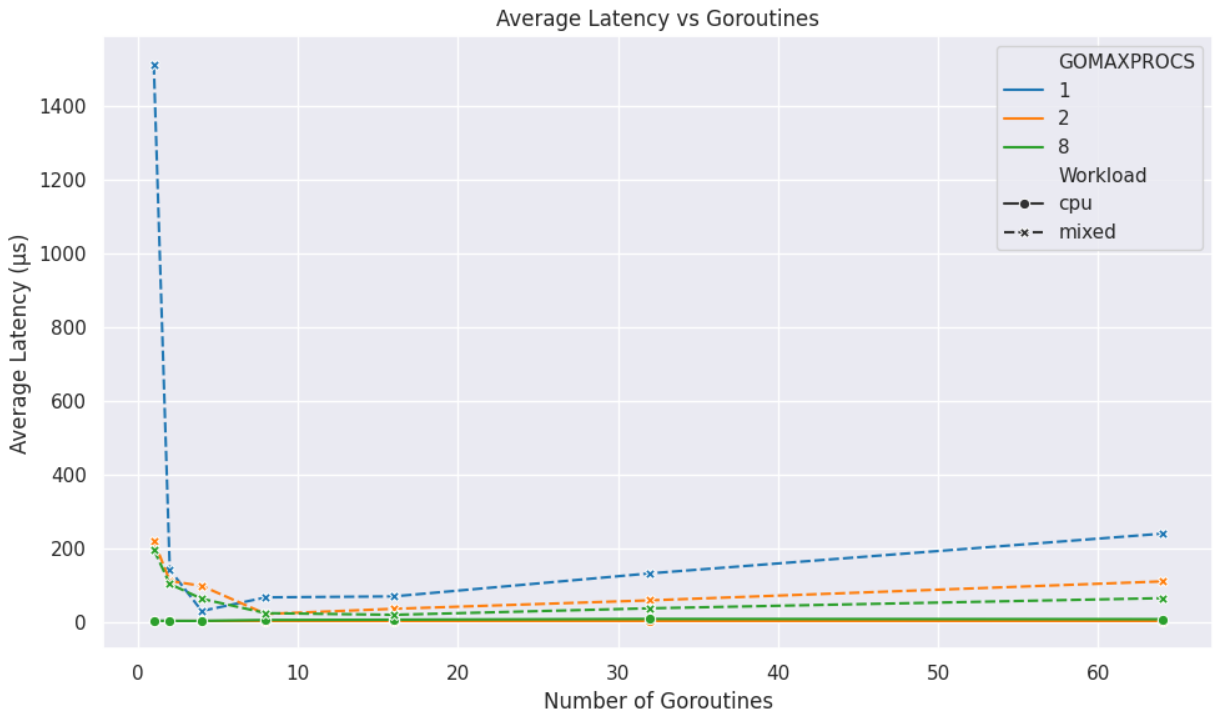
Work Load	GOMAXPROCS	Goroutines	Throughput	Total Time µs	Avg Latency µs	Std Latency µs
CPU	1	1	220294.66	9078.749	4.462	3.188
CPU	8	64	965905.18	132518.18	8.811	184.641
Mixed	1	1	685.215	2918793.9	1459.049	279.101
Mixed	8	64	550645.743	232454.35	63.415	568.255

نتایج مربوط به بار کاری cpu نشان می دهد که این بخش به شدت از افزایش GOMAXPROCS و تعداد goroutine ها بهره می برد. برای مثال در حالتی که تنها یک goroutine و GOMAXPROCS = 1 استفاده شده است، throughput حدود 220k تسک در ثانیه و میانگین latency حدود 4.46 میکروثانیه ثبت شده است. هنگامی که تعداد goroutine ها نیز بیشتر شده است، throughput به شکل قابل توجهی بالا می رود. نمونه‌ی مشخص آن حالتی است که GOMAXPROCS = 2 و تعداد goroutine ها 2 است که در آن throughput حدود 537k تسک در ثانیه و میانگین latency حدود 3.62 میکروثانیه گزارش شده است. در حالتی که GOMAXPROCS = 8 و تعداد goroutine ها 64 است، throughput بالایی دیده می شود و برای بخش CPU به حدود 966k تسک در ثانیه می رسد و میانگین latency هنوز در بازه‌ی تک رقمی باقی مانده است، هرچند انحراف معیار latency در این حالت بالا است.

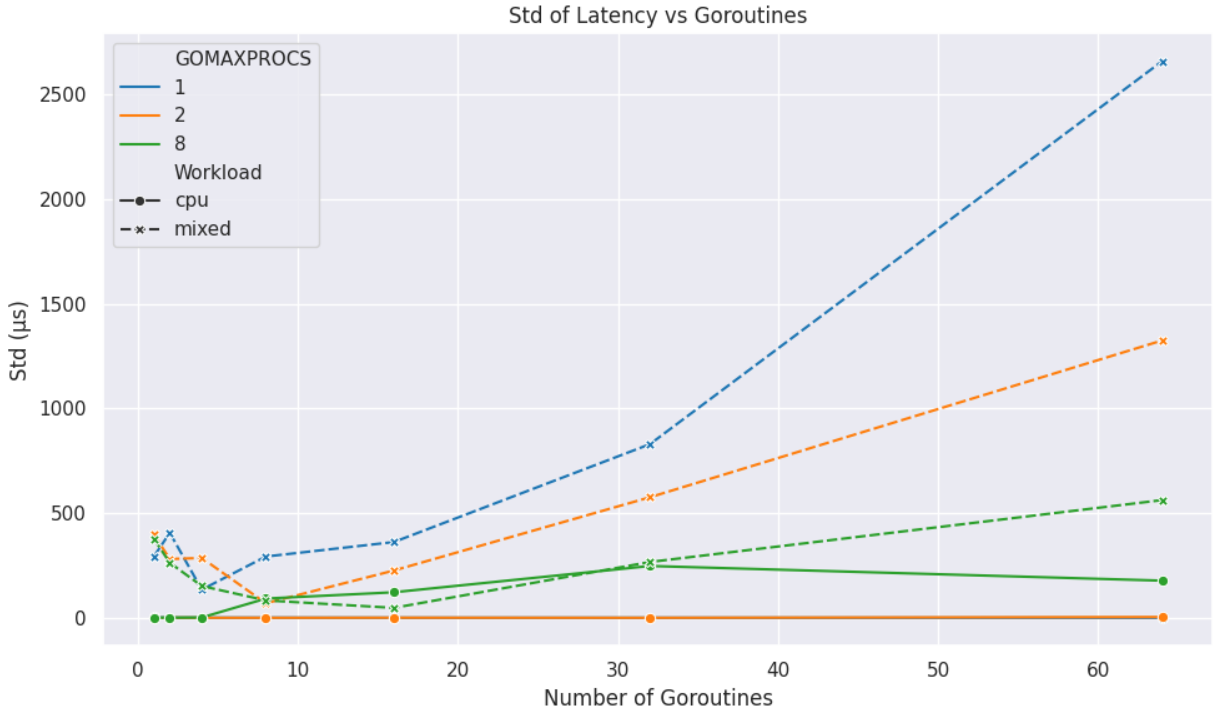
در بار کاری mixed وضعیت کمی متفاوت است. این بخش که شامل محاسبه، قفل‌گیری روی یک mutex مشترک و یک sleep کوتاه است، هم از نظر latency و هم از نظر throughput سنگین‌تر از بار CPU محض است. در ساده ترین حالت با GOMAXPROCS = 1 و تنها یک goroutine، throughput حدود 685 تسک در ثانیه و میانگین latency حدود 1459 میکروثانیه اندازه‌گیری شده است و انحراف معیار latency به حدود 279 میکروثانیه می رسد. این عدد در مقایسه با بخش CPU که latency تک رقمی داشت، نشان می دهد که اضافه شدن sleep و قفل‌گیری، تاثیر شدیدی بر زمان تکمیل هر تسک دارد. با افزایش تعداد goroutine ها، throughput برای بار mixed به طور کلی رشد می‌کند اما این رشد همواره یکنواخت و صعودی نیست. برای مثال در GOMAXPROCS = 1، throughput از حدود 8474 تسک در ثانیه در 2 goroutine به حدود 126837 در 8 goroutine می رسد، ولی با افزایش بیشتر تعداد goroutine ها به 16، throughput به حدود 72525 کاهش پیدا می کند و برای 64 goroutine به حدود 175390 می رسد.



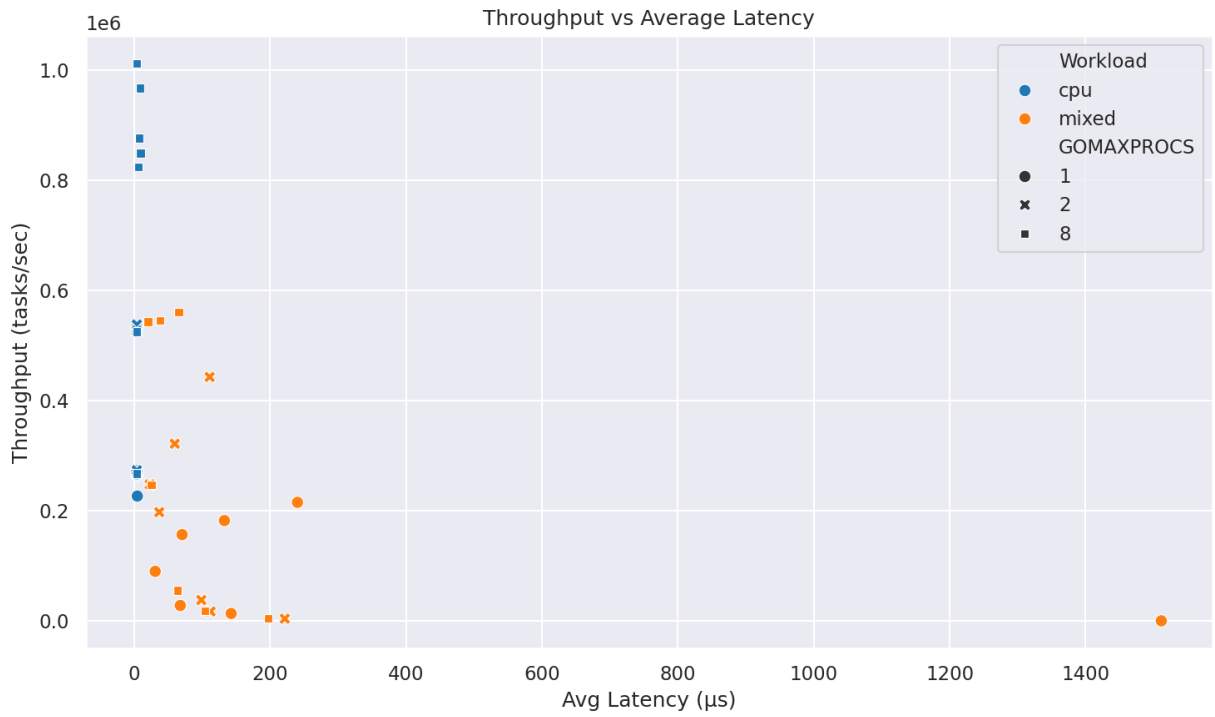
این نمودار نشان می دهد throughput چگونه با افزایش تعداد goroutine ها برای مقادیر مختلف GOMAXPROCS تغییر می کند. در بار cpu، وقتی GOMAXPROCS = 1 است، throughput بعد از چند goroutine به یک سطح تقریبا ثابت می رسد و افزایش بیشتر goroutine ها سود چندانی ندارد زیرا تنها یک thread در حال اجراست و اضافه شدن goroutine ها فقط هزینه ی زمان بندی را بالا می برد. با افزایش GOMAXPROCS، throughput با افزایش goroutine ها به طور قابل توجهی رشد می کند و در نهایت به حدود یک میلیون تسک بر ثانیه نزدیک می شود که نشان دهنده ی بهره گیری موثر از هسته های متعدد است. برای بار mixed، منحنی ها ناهموارتر هستند. throughput ابتدا با افزایش goroutine ها رشد می کند اما در برخی نقاط به‌ویژه با GOMAXPROCS کوچک افت هایی دیده می شود و سپس دوباره بالا می رود. این نوسان ها نشان می دهند که در حضور قفل و sleep، افزایش شدید تعداد goroutine ها می تواند سیستم را در برخی بازه ها باعث تضعیف عملکرد می شود.



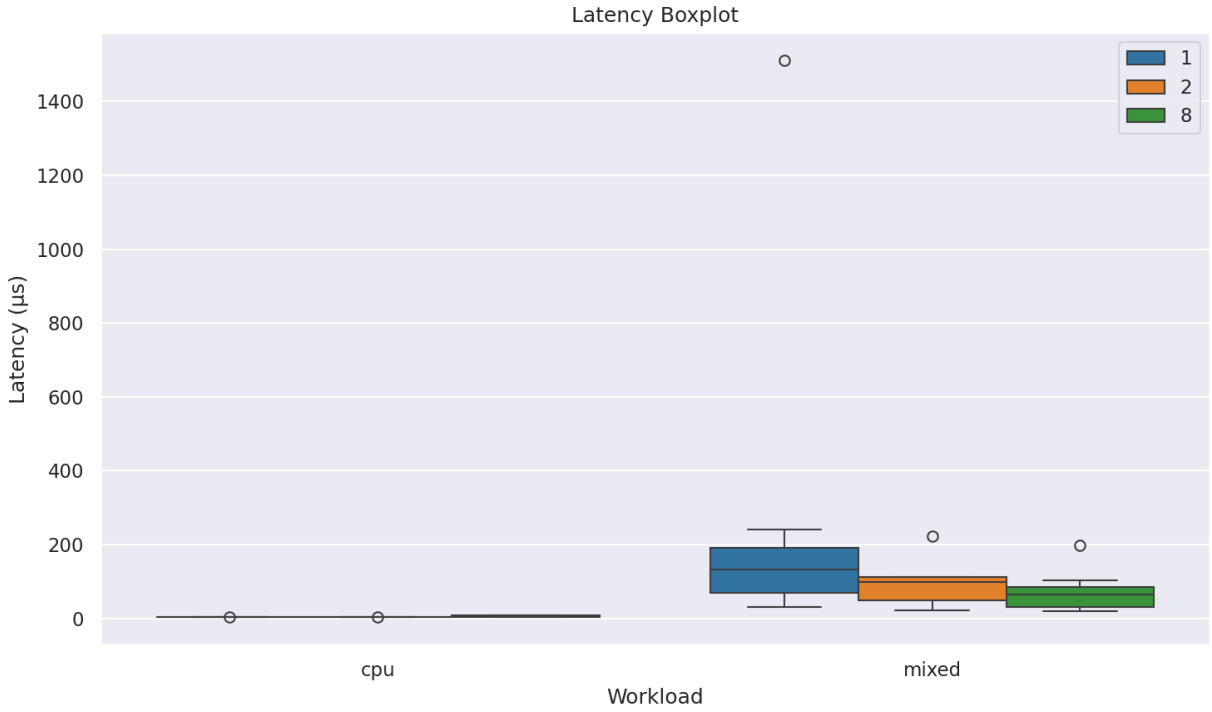
در این نمودار میانگین latency بر حسب تعداد goroutine ها رسم شده است. برای بار cpu منحنی ها در مقدار کمی ثابت می مانند و بیشتر در مقادیر بالای goroutine و GOMAXPROCS، کمی رو به افزایش می روند یعنی حتی با هم زمانی زیاد، latency متوسط برای cpu نسبت به mixed کوچک باقی می ماند. در بار mixed اما منحنی ها از یک نقطه‌ی بسیار بالا شروع می‌شوند و در گام های اولیه با افزایش goroutine ها کاهش محسوسی پیدا می کنند سپس در ناحیه ای پایدار می شوند و در مقادیر بزرگ goroutine دوباره رو به افزایش می روند. این الگو نشان می دهد که در ابتدا، افزودن چند goroutine می تواند latency متوسط را با بهبود بهره برداری از CPU کاهش دهد اما پس از یک حد مشخص، افزایش بیش از حد هم زمانی در بخش mixed منجر به صف های انتظار طولانی‌تر و context switching بیشتر شده و دوباره latency را بالا می برد.



این نمودار انحراف معیار latency را در برابر تعداد goroutine ها برای مقادیر مختلف GOMAXPROCS و دو نوع بار کاری نشان می دهد. برای بار cpu منحنی ها تقریبا پایین تر از mixed باقی می مانند و فقط در تنظیماتی با GOMAXPROCS بزرگ و تعداد goroutine های زیاد، کمی افزایش پیدا می کنند یعنی latency برای cpu همگن و قابل پیش‌بینی است. در مقابل منحنی های بار mixed از همان ابتدا مقادیر بالایی دارند و با افزایش تعداد goroutine ها، به‌خصوص برای GOMAXPROCS = 1 و 2 به شدت صعود می کنند و به بیش از دو هزار میکروثانیه می رسند. این رفتار نشان می دهد که در بار mixed با افزایش هم زمانی، پراکندگی latency به سرعت رشد می کند که نتیجه ی ترکیب contention روی mutex و افزایش context switching است.



در نمودار پراکندگی throughput نسبت به میانگین latency، نقاط مربوط به بار cpu در سمت چپ نمودار و نزدیک محور y متمرکز شده اند. latency کمتر از حدود 10 میکروثانیه است در حالی که throughput از حدود 220 هزار تا بیش از یک میلیون تسک بر ثانیه تغییر می کند. این یعنی سیستم در بارCPU محور می تواند throughput را بالا ببرد بدون آن‌که latency به طور معناداری افزایش یابد. در مقابل نقاط mixed در بازه ی latency های بزرگ تر پخش شده‌اند و throughput آن ها بین چند صد تا حدود نیم میلیون تسک بر ثانیه است. شکل کلی نمودار برای mixed نوعی trade off را نشان می‌دهد. هر جاthroughput بالاتر است، latency نیز نسبتا زیاد یا متغیر است. تفکیک قابل تشخیص دو خوشه ی cpu و mixed در این نمودار اثر نوع بار کاری بر تعادل بین throughput و latency را برجسته می کند.



این نمودار توزیع latency را برای دو نوع بار کاری و سه مقدار GOMAXPROCS شامل 1، 2 و 8 نشان می دهد. برای بار cpu جعبه ها بسیار کمتر از حالت mixed هستند و حتیoutlier ها هم در محدوده ی چند میکروثانیه باقی می مانند یعنی هم متوسط latency بسیار کوچک است و هم پراکندگی آن ناچیز است. در مقابل برای بار mixed جعبه ها در بازه های صد تا چند صد میکروثانیه قرار گرفته‌اند و چند outlier بزرگ دیده می شود. همچنین با افزایش GOMAXPROCS، میانه‌ی latency برای mixed کمی کاهش می یابد اما هنوز اختلاف مرتبه‌ای آن با cpu حفظ می شود. این نمودار به‌خوبی نشان می دهد که اضافه شدن عملیات قفل‌گیری و sleep در حالت mixed باعث افزایش مقدار و هم واریانس latency می شود.

سوالات تحلیلی

1. با افزایش تعداد goroutine ها چه اتفاقی برای زمان اجرا می افتد؟

رفتار زمان اجرا با افزایش تعداد goroutine ها به نوع workload و مقدارGOMAXPROCS بستگی دارد. در workload نوع cpu، وقتی GOMAXPROCS = 1 است، بعد از حدود ۸ goroutine زمان اجرای کل تقریبا ثابت می شود چون فقط یک هسته درگیر است وgoroutine های بیشتر فقط هزینه ی زمان بندی را بالا می برند. در مقابل وقتی GOMAXPROCS برابر تعداد هسته ها تنظیم می شود، افزایش تعدادgoroutine ها تا حدی باعث کاهش قابل توجه زمان اجرای کل و افزایش throughput می شود. در workload نوع mixed ابتدا با افزایش goroutine ها زمان اجرا کاهش پیدا می کند، اما بعد از یک نقطه افزایش بیشتر goroutine ها باعث طولانی تر شدن زمان اجرا می شود که نشان دهنده ی افزایش overhead هم زمانی است.

2. آیا افزایش تعداد goroutine ها همیشه باعث افزایش throughput می شود؟

خیر نتایج نشان می دهد که افزایش تعداد goroutine ها همیشه به افزایش throughput منجر نمی شود. در workload cpu با GOMAXPROCS = 1، throughput بعد از حدود goroutine8 به اشباع می رسد و افزایش بیشتر goroutine ها تاثیر محسوسی ندارد. درworkload mixed، throughput در ابتدا با افزایش goroutine ها رشد می کند اما در برخی ترکیب ها مثلا با GOMAXPROCS کم و goroutine های بالا throughput حتی کاهش پیدا کرده است. این رفتار نشان می دهد که پس از یک حد مشخص، هزینه ی زمان بندی بیشتر از سود هم زمانی است و throughput را محدود یا کاهش می دهد.

3. تفاوت رفتار برنامه در GOMAXPROCS = 1 و GOMAXPROCS = NumCPU چیست؟

در حالت GOMAXPROCS = 1، تمام goroutine ها روی یک هسته اجرا می شوند بنابراین برنامه موازی سازی واقعی ندارد. در این حالت افزایش goroutine ها بعد از یک حد مشخص فقط زمان بند را شلوغ تر می‌کند. وقتی GOMAXPROCS = NumCPU تنظیم می شود، زمان‌بند Go می‌تواند goroutine ها را روی چند هسته به صورت هم‌زمان اجرا کند در نتیجه در workload cpu، throughput به‌طور چشمگیری افزایش پیدا می کند و استفاده از CPU ها بهتر می شود.

4. در کدام نوع workload اثرات زمان بندی و switching بیشتر دیده می شود؟

بر اساس نتایج و نمودارهای latency، اثرات زمان‌بندی و context switching در workload نوع mixed واضح تر است. این workload شامل ترکیب عملیات محاسباتی، قفل‌گیری روی یک mutex مشترک و sleep است بنابراین goroutine ها مرتبا وارد حالت انتظار می شوند و زمان بند Go باید بین goroutine های مختلف جا به جا شود. نتیجه ی این وضعیت، latency بالاتر، انحراف معیار بزرگ تر و نوسانات بیشتر در throughput است. در مقابل در workload cpu که goroutine ها عمدتا مشغول محاسبه ی پیوسته هستند، latency پایین تر و پایدار تر است و اثرات switching کمتر خود را نشان می دهد مگر در مقادیر بسیار بالای هم زمانی.

5. از چه نقطه ای به بعد افزایش هم زمانی مفید نیست یا حتی کارایی را کاهش می دهد؟

به طور کلی زمانی که overhead هم‌زمانی مثل context switching، صف‌های انتظار، قفل‌گیری روی mutex و ... از سود اجرای هم زمان بیشتر شود آنگاه افزایش goroutine دیگر مفید نیست و حتی می تواند کارایی را کاهش دهد. در نتایج این آزمایش ها، برای workload cpu با GOMAXPROCS = 1 and 2 این نقطه تقریبا بعد از حدود 8 goroutine دیده می شود جایی که throughput تقریبا ثابت می ماند و latency کمی افزایش پیدا می‌کند. در workload mixed این نقطه شدید تر ظاهر می شود. در برخی ترکیب‌ها، افزایش goroutine ها به خصوص با GOMAXPROCS پایین باعث کاهش throughput و افزایش latency و انحراف معیار آن شده است. این رفتار نشان می دهد که بعد از یک حد مشخص هم زمانی، سیستم اشباع می شود و goroutine های بیشتر نه تنها کمکی نمی کنند بلکه عملکرد را تضعیف می کنند.

بخش سوم: ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی

مقدمه

در این بخش از تمرین، هدف تبدیل ایده ی سرویس محاسباتی بخش های قبل به یک سرویس شبکه ای ساده است. در این بخش به جای این که ارتباط از طریق Pipe یا ورودی ترمینال انجام شود، یک سرویس HTTP با پیاده سازی شد که درخواست های محاسباتی را از طریق آدرس های وب دریافت می کند و پاسخ را در قالب JSON برمی گرداند. سپس این سرویس با استفاده از Docker کانتینری شد و داخل ماشین مجازی اجرا گردید. در نهایت از سیستم Host به این سرویس متصل شدیم تا نشان داده شود سرویس داخل VM و داخل کانتینر از بیرون قابل دسترسی است.

در این پیاده سازی دو endpoint اصلی وجود دارد. endpoint اول به نام /health برای بررسی سلامت سرویس استفاده می شود و endpoint دوم به نام /compute برای انجام محاسبات عددی به کار می رود. سرویس از عملیات هایadd ، sub، mul، div و همچنین چند عملیات اضافی مانند mod و pow پشتیبانی می کند. پاسخ های سرویس هم در حالت موفقیت و هم در حالت خطا به صورت JSON تولید می شوند تا خروجی ساختارمند و قابل پردازش باشد.

بخش اصلی برنامه با استفاده از پکیج net/http ساخته شده است. در تابعmain ، ابتدا مسیرهای مورد نیاز با استفاده از یک router ساده ثبت می شوند. مسیر /health به تابع healthHandler و مسیر /compute به تابع computeHandler متصل شده است. سپس برنامه با استفاده از ListenAndServe روی پورت 8080 اجرا می شود.

در endpoint مربوط بهhealth ، اگر درخواست با متد GET ارسال شده باشد، سرویس یک پاسخ JSON شامل وضعیت سرویس، نام سرویس و زمان فعلی برمی گرداند. این endpoint برای این استفاده می شود که بتوانیم به سرعت بررسی کنیم آیا سرویس داخل کانتینر به درستی اجرا شده است یا خیر.

در endpoint مربوط بهcompute ، پارامترهای درخواست از query string خوانده می شوند. در این پیاده سازی پارامتر operator برای نام عملیات و پارامترهای a و b برای دو عدد ورودی استفاده شده اند. برای مثال درخواست زیر به معنای جمع دو عدد 10 و 20 است:

```
curl "http://localhost:8080/compute?operator=add&a=10&b=20"
```

پس از دریافت درخواست، برنامه ابتدا بررسی می کند که پارامترهای مورد نیاز وجود داشته باشند. سپس مقدارهای a و b به عدد تبدیل می شوند. اگر تبدیل با خطا مواجه شود، پاسخ خطا تولید می شود. بعد از آن نوع عملیات بررسی شده و محاسبه انجام می شود. در صورت موفقیت، نتیجه در قالب JSON به کاربر برگردانده می شود.

مدیریت خطا در سرویس

در این بخش مدیریت خطا اهمیت زیادی دارد زیرا سرویس HTTP ممکن است درخواست های نامعتبر دریافت کند. به همین دلیل چند حالت خطا در برنامه بررسی شده است. اگر یکی از پارامترهای لازم وجود نداشته باشد، سرویس پاسخ خطای مناسب برمی گرداند. اگر مقدارهای a یا b عددی نباشند، برنامه آن ها را نامعتبر تشخیص می دهد. اگر نام عملیات پشتیبانی نشده باشد، خطای operation نامعتبر تولید می شود. همچنین در عملیات تقسیم، تقسیم بر صفر به صورت جداگانه بررسی شده و در این حالت سرویس به جای متوقف شدن، یک پاسخ JSON شامل پیام خطا برمی گرداند.

علاوه بر این، اگر کاربر به جای متد GET از متد دیگری مانند POST استفاده کند، سرویس خطای مربوط به متد نادرست را برمی گرداند. این موضوع باعث می شود رفتار سرویس مشخص و قابل پیش بینی باشد و در برابر ورودی های اشتباه به صورت مناسب عمل کند.

کانتینری سازی سرویس با Docker

پس از پیاده سازی سرویس، مرحله بعدی کانتینری سازی آن با Docker بود. برای این کار یک Dockerfile نوشته شد. در Dockerfile ابتدا از یک image رسمی Go برای build کردن برنامه استفاده می شود. در این مرحله فایل های پروژه داخل محیط Docker کپی می شوند و برنامه Go با دستور go build کامپایل می شود. خروجی این مرحله یک فایل اجرایی است.

سپس در مرحله دوم Dockerfile ، از یک image سبک تر استفاده می شود و فقط فایل اجرایی نهایی به آن منتقل می گردد. این روش باعث می شود image نهایی کوچک تر و تمیزتر باشد، زیرا دیگر نیازی نیست کامپایلر Go و فایل های اضافی داخل image نهایی وجود داشته باشند. در پایان Dockerfile مشخص می کند که سرویس روی پورت 8080 اجرا می شود و هنگام شروع کانتینر، فایل اجرایی سرویس اجرا شود.

برای ساخت image از دستور زیر استفاده شد:

```
sudo docker build -t hw1-part3-service .
```

پس از اجرای این دستور، image مربوط به سرویس با نام hw1-part3-service ساخته شد. خروجی دستور docker images نشان می دهد که image با موفقیت ساخته شده و در لیست image های Docker قرار گرفته است.

```
[~/myFolder/distributed/CA1/HW1/part3]
mahdi  sudo docker images
[sudo] password for mahdi:
REPOSITORY          TAG       IMAGE ID       CREATED
hw1-part3-service   latest    a902dbee268b   17 hours ago
```

اجرای کانتینر داخل ماشین مجازی

بعد از ساخته شدنimage ، کانتینر با استفاده از دستور زیر اجرا شد:

```
sudo docker run -d --name hw1-part3-container -p 8080:8080 hw1-part3-service
```

در این دستور، گزینه d- باعث می شود کانتینر در پس زمینه اجرا شود. گزینه name-- یک نام مشخص برای کانتینر تعیین می کند تا بعدا بتوانیم راحت تر آن را مدیریت کنیم. بخش مهم این دستور گزینه p 8080:8080- است. این گزینه پورت 8080 ماشین مجازی را به پورت 8080 داخل کانتینر متصل می کند. در نتیجه اگر در داخل VM یا از بیرون VM درخواستی به پورت 8080 ارسال شود، این درخواست به سرویس داخل کانتینر منتقل می شود.

پس از اجرای کانتینر، Docker یک container ID تولید کرد که نشان دهنده شروع موفق کانتینر است:

```
[~/myFolder/distributed/CA1/HW1/part3]
mahdi  sudo docker run -d --name hw1-part3-container -p 8080:8080 hw1-part3-service

15e2edf6f7ad73f8440e34ecdd1953eb4bf8f3619ee0f885bd787df38390666b
```

برای بررسی وضعیت کانتینر از دستور زیر استفاده شد:

```
sudo docker ps
```

خروجی این دستور نشان داد که کانتینر hw1-part3-container در حال اجراست و پورت 8080 به درستی map شده است. وجود عبارت نشان داده شده در خروجی نشان می دهد که پورت 8080 ماشین مجازی به پورت 8080 کانتینر متصل شده است.

```
mahdi  sudo docker ps
[sudo] password for mahdi:
CONTAINER ID   IMAGE              COMMAND      CREATED              STATUS              PORTS                                         NAMES
15e2edf6f7ad   hw1-part3-service  "./server"   About a minute ago   Up About a minute   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp     hw1-part3-container
```

همچنین برای مشاهده log های سرویس از دستور زیر استفاده شد:

sudo docker logs hw1-part3-container

خروجی این دستور نشان داد که سرویس HTTP روی پورت 8080 اجرا شده است. این موضوع تایید می کند که برنامه Go داخل کانتینر به درستی شروع شده و آماده دریافت درخواست است.

```
mahdi  sudo docker logs hw1-part3-container

2026/05/11 14:09:18 HTTP service is running on port 8080
```

آزمایش سرویس داخل ماشین مجازی

در مرحله بعد، ابتدا سرویس از داخل خود ماشین مجازی تست شد. برای بررسی endpoint مربوط به health از دستور زیر استفاده شد:

curl http://localhost:8080/health

پاسخ دریافتی یک JSON شامل وضعیت ok ، نام سرویس و زمان اجرای درخواست بود. این خروجی نشان می دهد که سرویس داخل کانتینر از داخل VM قابل دسترسی است.

```
mahdi  curl http://localhost:8080/health
{"status":"ok","service":"Distributed Systems HW1 Part3","time":"2026-05-11T14:14:21Z"}
```

سپس endpoint مربوط به compute با دستور زیر آزمایش شد:

```
[~]
mahdi  curl "http://localhost:8080/compute?operator=add&a=10&b=20"

{"operation":"add","a":10,"b":20,"result":30}
```

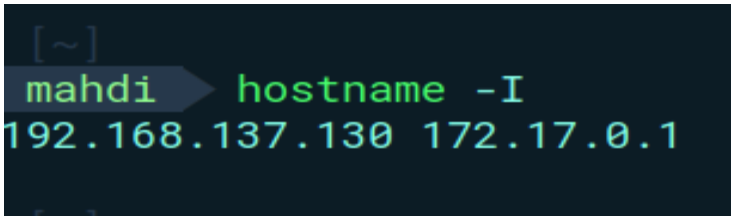
این خروجی نشان می دهد که سرویس توانسته پارامترهای ارسالی را دریافت کند، عملیات add را تشخیص دهد، دو عدد ورودی را جمع کند و نتیجه را در قالب JSON برگرداند.

اتصال از Host به سرویس داخل VM

پس از اطمینان از اجرای صحیح سرویس داخل ماشین مجازی، مرحله بعدی اتصال از سیستم Host به سرویس داخل VM بود. برای این کار ابتدا آدرس IP ماشین مجازی با دستور زیر به دست آمد:

```
hostname -I
```

خروجی این دستور دو آدرس IP را نشان داد. آدرس 192.168.137.130 مربوط به شبکه VM بود و برای اتصال از Host به VM استفاده شد. آدرس 172.17.0.1 نیز مربوط به شبکه داخلی Docker است و برای اتصال از Host مناسب نیست.

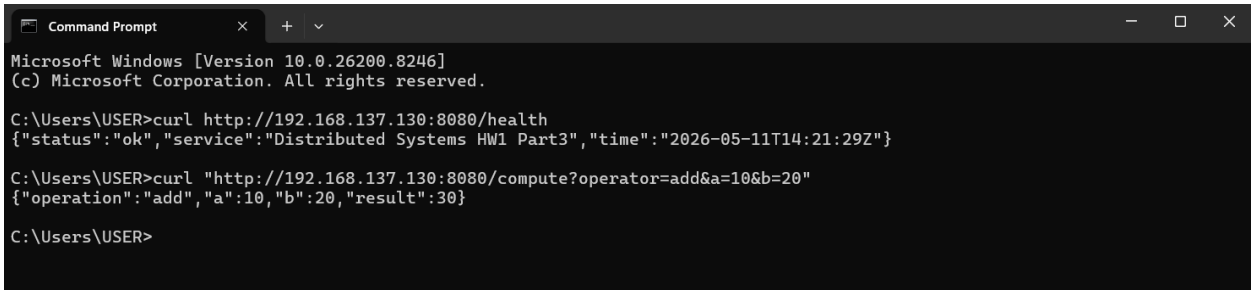


سپس از سیستم Host که در این آزمایش Windows بود، با استفاده از Command Prompt به endpoint مربوط به health درخواست ارسال شد:



پاسخ JSON دریافتی نشان داد که Host توانسته است به سرویس داخل VM متصل شود. این موضوع نشان می دهد که مسیر ارتباطی Host به VM و سپس از VM به کانتینر به درستی کار می کند.

در ادامه endpoint مربوط به compute نیز از سمت Host آزمایش شد:



این خروجی نشان می دهد که درخواست از سیستم Host به ماشین مجازی ارسال شده، سپس از طریق port mapping به کانتینر Docker رسیده و در نهایت سرویس Go داخل کانتینر پاسخ مناسب را برگردانده است.

جمع بندی

در این بخش یک سرویس HTTP ساده با زبان Go پیاده سازی شد که دو endpoint اصلی /health و /compute را پشتیبانی می کند. سرویس برای درخواست های محاسباتی، پارامترهای ورودی را از query string دریافت کرده و نتیجه را به صورت JSON برمی گرداند. همچنین خطاهایی مانند پارامترهای ناقص، ورودی غیرعددی، عملیات نامعتبر، تقسیم بر صفر و متد HTTP نادرست مدیریت شده اند. پس از پیاده سازی، سرویس با استفاده از Docker کانتینری شد Docker image .مربوط به برنامه ساخته شد و سپس کانتینر داخل ماشین مجازی اجرا گردید. با استفاده از port mapping ، پورت 8080 کانتینر به پورت 8080 ماشین مجازی متصل شد. سپس ابتدا سرویس از داخل VM و بعد از آن از سیستم Host آزمایش شد. خروجی های به دست آمده نشان دادند که سرویس به درستی داخل کانتینر اجرا شده و Host می تواند از طریق IP ماشین مجازی به آن دسترسی داشته باشد.